

TP 4 - Bases de Datos NoSQL con Redis

Sección 1 - Introducción a Redis

1. ¿Qué tipo de base de datos es Redis? ¿En qué se diferencia de una base de datos relacional y de otras bases de datos NoSQL como MongoDB?

Redis (Remote Dictionary Server) es una base de datos **NoSQL en memoria, de tipo clave-valor**.

Diferencias con una Base de Datos Relacional (RDBMS)

- **Modelo de Datos:** Mientras que los RDBMS organizan la información en **tablas, filas y columnas** con un esquema fijo, Redis utiliza un modelo de **clave-valor** donde el "valor" puede ser una estructura de datos (como strings, listas o hashes) sin un esquema predefinido.
- **Almacenamiento:** Las bases relacionales están diseñadas para ejecutarse principalmente en **disco**. Redis almacena los datos en **memoria RAM**, lo que le otorga una velocidad superior pero requiere mecanismos adicionales (como RDB o AOF) para garantizar la persistencia ante un apagado del sistema.
- **Relaciones y Consultas:** Los RDBMS permiten realizar **JOINS** y consultas complejas sobre múltiples tablas. Redis, al ser un almacén de claves, no soporta de forma nativa relaciones complejas ni integridad referencial automática; la lógica de vinculación debe ser manejada por la aplicación.

Diferencias con MongoDB (NoSQL Documental)

- **Tipo de Modelo NoSQL:** MongoDB es una base de datos **orientada a documentos** (almacena objetos tipo JSON/BSON), mientras que Redis es un **almacén de clave-valor**.
- **Ubicación de los Datos:** MongoDB reside principalmente en **disco** (usando la memoria como caché), lo que la hace adecuada para grandes volúmenes de datos persistentes. Redis reside **íntegramente en memoria**, lo que limita su capacidad al tamaño de la RAM disponible pero reduce drásticamente la latencia.
- **Capacidad de Consulta:** En MongoDB se pueden realizar consultas basadas en los **campos internos** de un documento y crear índices sobre ellos. En un almacén de clave-valor tradicional, el acceso se realiza primordialmente por su **clave**, aunque Redis permite cierta inspección al soportar estructuras como conjuntos o listas dentro del valor.
- **Casos de Uso:** Redis es ideal para datos transitorios que requieren acceso instantáneo, como **sesiones de usuario, carritos de compra o rankings** en tiempo real. MongoDB se utiliza para modelos de dominio más complejos que requieren persistencia duradera y consultas más ricas sobre la estructura de los datos.

2. ¿Dónde almacena los datos Redis? ¿Qué implicancias tiene esto en términos de velocidad y de persistencia?

Redis almacena sus datos primordialmente en la **memoria principal (RAM)**. Esta naturaleza de funcionamiento "en memoria" define sus características fundamentales:

Implicancias en términos de Velocidad

- **Rendimiento Extremo:** Al residir en la memoria volátil, Redis elimina la necesidad de realizar operaciones de **E/S de disco (I/O)** para el procesamiento de datos, lo que le otorga una ventaja de rendimiento sustancial frente a bases de datos que dependen del disco.
- **Baja Latencia:** Permite una respuesta mucho más rápida a las solicitudes, siendo ideal para aplicaciones en tiempo real, motores de estructuras de datos y como sistema de caché.
- **Eficiencia en el Procesamiento:** La manipulación de datos directamente en memoria simplifica la arquitectura interna, permitiendo que un procesador de eventos pueda lograr un rendimiento impresionante con latencia mínima.

Implicancias en términos de Persistencia

- **Volatilidad:** El mayor riesgo de este almacenamiento es que la memoria RAM es volátil; se pierden todos los datos si se produce un corte de energía, un fallo en el sistema operativo o si el servidor se apaga.
- **Compromiso de Durabilidad:** Si Redis se utiliza estrictamente en memoria para ganar velocidad, cualquier actualización realizada desde el último volcado a disco (en caso de existir) se perderá si el servidor falla.
- **Mecanismos de Respaldo:** Aunque su caso de uso principal no es la persistencia en disco, Redis ofrece mecanismos para mitigar la pérdida de datos, como **RDB** (instantáneas del conjunto de datos) y **AOF** (un registro de cada operación de escritura), que permiten reconstruir el estado de la base de datos tras un reinicio.
- **Limitación de Espacio:** A diferencia del almacenamiento en disco (backing store), que es más grande y económico, la capacidad de Redis está estrictamente limitada por el tamaño de la memoria RAM disponible en el servidor.

3. ¿Qué tipos de datos soporta Redis? Listar y describir brevemente cada uno.

Tipos de datos que soporta Redis:

- **Strings:** Es el tipo más básico y puede contener cualquier tipo de datos, como texto o valores numéricos. Permiten operaciones como incremento o decremento atómico sobre números.
- **Listas (Lists):** Colecciones de strings ordenadas por el orden de inserción. Permiten agregar elementos tanto por el extremo izquierdo como por el derecho.

- **Conjuntos (Sets):** Colecciones no ordenadas de strings únicos. Soportan operaciones de lógica de conjuntos como unión, intersección y diferencia.
- **Conjuntos Ordenados (Sorted Sets / ZSets):** Similares a los conjuntos, pero cada elemento tiene asociado un "score" numérico que se utiliza para mantener los elementos ordenados. Son la base para el almacenamiento de **datos geoespaciales** en Redis.
- **Hashes:** Mapas entre campos de strings y valores de strings, lo que los hace ideales para representar objetos (por ejemplo, un perfil de usuario con múltiples atributos).
- **Streams:** Estructuras diseñadas para el procesamiento de eventos y flujo de datos en tiempo real.
- **JSON:** Permite almacenar, actualizar y consultar documentos en formato JSON de manera nativa.
- **Soporte Geoespacial:** Permite almacenar coordenadas (longitud y latitud) para realizar consultas de proximidad o calcular distancias entre puntos.
- **Vectores (Vectors):** Utilizados para búsquedas vectoriales, especialmente en aplicaciones de inteligencia artificial y modelos de lenguaje (LLMs).

4. Enunciar las características principales de Redis.

Sus características principales son:

- **Almacenamiento en memoria (In-memory):** A diferencia de las bases de datos tradicionales que residen en disco, Redis almacena los datos primordialmente en la **memoria RAM**. Esto le permite ofrecer un rendimiento extremo con una latencia mínima, siendo ideal para aplicaciones que requieren respuestas en tiempo real.
- **Servidor de estructuras de datos:** Aunque técnicamente es un almacén de **clave-valor**, a menudo se le denomina "servidor de estructuras de datos" porque los valores no son simples cadenas de texto, sino que pueden ser estructuras complejas como **listas, conjuntos (sets), conjuntos ordenados, hashes, streams, JSON y vectores**.
- **Versatilidad de roles:** Gracias a su velocidad y soporte de estructuras, Redis se utiliza ampliamente en tres roles fundamentales: como **base de datos**, como sistema de **caché** para aliviar la carga de bases de datos en disco, y como **broker de mensajes** para la comunicación entre aplicaciones.
- **Persistencia opcional:** Aunque su fuerte es el funcionamiento en memoria (que es volátil), Redis ofrece mecanismos para garantizar la durabilidad de los datos mediante volcados a disco (**RDB**) o registros de operaciones (**AOF**), permitiendo la recuperación tras un reinicio.
- **Capacidades avanzadas de consulta:** A diferencia de otros almacenes de clave-valor simples, Redis permite realizar operaciones sofisticadas sobre sus estructuras, como intersecciones de conjuntos, rangos de búsqueda y soporte nativo para **datos geoespaciales** (coordenadas de longitud y latitud).

- **Modelo sin esquema (Schemaless):** Como parte de la familia NoSQL, Redis no requiere la definición previa de tablas o columnas fijas, permitiendo una gran flexibilidad en el desarrollo de aplicaciones.
 - **Código abierto (Open Source):** Es una tecnología accesible y ampliamente adoptada por la comunidad, con múltiples bibliotecas cliente para diferentes lenguajes de programación.
5. Comparar Redis con los RDBMS: ¿en qué casos conviene usar Redis en lugar de una base de datos relacional y en cuáles no?

¿Cuándo conviene usar Redis en lugar de una base de datos relacional? Redis es la opción preferida cuando la **velocidad de acceso** es el factor más crítico y los datos no requieren una estructura altamente relacionada:

- **Caching (Alivio de carga):** Se utiliza para almacenar resultados de consultas frecuentes que, en un RDBMS, requerirían costosos **JOINS** o procesamiento de disco, reduciendo drásticamente la latencia.
- **Datos transitorios o volátiles:** Es ideal para información que no necesita persistencia a largo plazo o que cambia con mucha frecuencia, como **sesiones de usuario, carritos de compra** o tokens de autenticación.
- **Analíticas y ránking en tiempo real:** Gracias a sus estructuras como los **Sorted Sets**, Redis permite gestionar contadores y tablas de posiciones (leaderboards) con actualizaciones constantes de forma mucho más eficiente que un RDBMS.
- **Sistemas de mensajería (Pub/Sub):** Redis funciona excelentemente como un **broker de mensajes** para la comunicación entre diferentes partes de una aplicación distribuida.
- **Operaciones atómicas simples:** Cuando se requieren incrementos numéricos o manipulaciones de listas/conjuntos de forma atómica y ultrarrápida.

¿Cuándo NO conviene usar Redis (y es mejor un RDBMS)? No se recomienda el uso de Redis como base de datos principal en los siguientes escenarios:

- **Relaciones complejas de datos:** Si la aplicación depende fuertemente de la integridad referencial y requiere realizar múltiples **vínculos (JOINS)** entre diferentes entidades para obtener información, Redis resulta ineficiente ya que estas relaciones deben gestionarse manualmente en la lógica de la aplicación.
- **Garantías ACID estrictas:** Cuando se necesitan transacciones complejas que involucren múltiples registros y tablas con un cumplimiento riguroso de atomicidad y consistencia global (ej. sistemas contables o financieros críticos), el modelo de transacciones de un RDBMS es muy superior.
- **Grandes volúmenes de datos persistentes:** Redis almacena todo en la **memoria RAM**, la cual es mucho más costosa y limitada en espacio que el almacenamiento en disco de los

RDBMS. No es apto para almacenar terabytes de datos históricos que no caben en la memoria del servidor.

- **Consultas Ad-hoc ricas:** Si se necesita filtrar o buscar datos por múltiples campos internos que no son la "clave", un RDBMS (o una documental como MongoDB) ofrece capacidades de indexación y consulta mucho más potentes. En Redis, el acceso es primordialmente por su clave única.
- **Máxima durabilidad garantizada:** Aunque Redis tiene mecanismos de persistencia (RDB/AOF), su naturaleza en memoria implica que siempre existe un pequeño riesgo de pérdida de datos ante un fallo repentino del servidor, algo que los RDBMS minimizan al escribir directamente en disco de forma transaccional

6. ¿Redis tiene soporte para transacciones? ¿Cómo funcionan? ¿Qué garantías ofrecen y qué limitaciones tienen respecto de las transacciones ACID?

Sí, **Redis tiene soporte para transacciones**, aunque su funcionamiento y garantías difieren del modelo tradicional de las bases de datos relacionales debido a su arquitectura orientada a objetos y su naturaleza NoSQL.

- **Funcionamiento: Granularidad por Agregación:** En las bases de datos NoSQL como Redis, las transacciones operan bajo una **granularidad distinta** a la de los sistemas relacionales.
 - **Nivel de Agregado:** Redis es una base de datos orientada a agregaciones, donde la unidad de información es el par **clave-valor**.
 - **Atomicidad Local:** Las transacciones se garantizan a nivel de un **único agregado**. Esto significa que cualquier operación sobre un documento o clave (aunque contenga estructuras complejas como listas o hashes) se ejecuta como una unidad indivisible que tiene éxito o falla en su totalidad.
- **Garantías ofrecidas**
 - **Atomicidad en el Agregado:** Se asegura que las operaciones sobre una misma clave no sean interferidas por otros procesos.
 - **Consistencia Lógica:** Al tratar el agregado como una unidad, se garantiza que los datos internos de esa clave tengan sentido entre sí.
 - **Velocidad:** Al limitar el alcance de la transacción, Redis mantiene su capacidad de ofrecer baja latencia y alto rendimiento.
- **Limitaciones respecto de las transacciones ACID:** El modelo de Redis presenta limitaciones claras cuando se compara con el estándar **ACID** (Atómico, Consistente, Aislado y Duradero) de los RDBMS:
 - **Falta de transacciones multi-agregado:** A diferencia de los RDBMS, que permiten manipular cualquier combinación de filas en múltiples tablas en una sola transacción, Redis generalmente **no soporta transacciones que abarquen múltiples claves** de

forma atómica. Si una operación requiere modificar varias claves, la aplicación debe gestionar manualmente la posibilidad de un fallo parcial.

- **Compromiso de Durabilidad (D):** Redis almacena los datos primordialmente en la memoria RAM para ganar velocidad. Esto implica un riesgo para la durabilidad: si el servidor falla antes de que los cambios se vuelquen al disco mediante los mecanismos de persistencia (RDB o AOF), los datos de las últimas transacciones podrían perderse.
- **Teorema CAP:** En entornos de cluster (distribuidos), Redis debe sopesar la **Consistencia** frente a la **Disponibilidad** y la **Tolerancia a Particiones**. A menudo se relaja la consistencia fuerte para asegurar que el sistema siga operativo, lo que se aleja del cumplimiento estricto de ACID.

7. ¿Redis tiene persistencia? Describir los mecanismos disponibles (RDB y AOF) e indicar las diferencias entre ellos.

Sí, **Redis tiene persistencia**, aunque su funcionamiento principal es en memoria RAM. Para mitigar el riesgo de pérdida de datos debido a la volatilidad de la memoria, ofrece dos mecanismos principales: **RDB** y **AOF**.

1. **RDB (Redis Database):** Este mecanismo consiste en realizar **instantáneas (snapshots)** del conjunto de datos en momentos específicos.
 - **Funcionamiento:** Redis realiza un volcado de toda la información que tiene en memoria a un archivo binario en el disco de forma compacta.
 - **Ventajas:** Es ideal para realizar copias de seguridad (backups) y permite un reinicio del servidor muy rápido, ya que solo debe cargar el archivo de datos final en memoria.
 - **Desventajas:** Existe un riesgo de pérdida de datos mayor, ya que si el servidor falla, se perderá toda la información generada desde la última instantánea realizada.
2. **AOF (Append Only File):** Este mecanismo se basa en mantener un **registro (log) de cada operación de escritura** recibida por el servidor.
 - **Funcionamiento:** Cada comando que modifica los datos (como `SET` o `DEL`) se añade al final de un archivo de texto. Al reiniciar el servidor, Redis "reproduce" todos estos comandos para reconstruir el estado de la base de datos.
 - **Ventajas:** Ofrece una durabilidad mucho mayor y una pérdida de datos mínima, ya que las operaciones se registran casi en tiempo real.
 - **Desventajas:** El archivo resultante suele ser mucho más grande que el de RDB y la recuperación de datos tras un reinicio es más lenta, pues el sistema debe procesar cada instrucción guardada en el log una por una.

Diferencias principales

Característica	RDB	AOF
Método	Instantáneas periódicas (Snapshots).	Registro continuo de comandos (Log).
Seguridad de datos	Menor (se pierde lo ocurrido desde el último volcado).	Mayor (registra casi cada operación).
Tamaño de archivo	Compacto y pequeño.	Generalmente más grande.
Velocidad de carga	Muy rápida al reiniciar.	Más lenta (debe re-ejecutar comandos).
Uso principal	Backups y recuperación rápida.	Máxima integridad y durabilidad.

8. ¿Cuáles son los principales casos de uso de Redis en aplicaciones reales?

Principales casos de uso en aplicaciones reales:

- **Gestión de Sesiones (Session Store):** Es uno de los usos más comunes. Almacenar estados de sesión de usuarios web en Redis es mucho más rápido que hacerlo en un RDBMS o en disco, ya que todo el objeto de sesión se recupera con un solo comando `GET` mediante el `sessionID`.
- **Sistemas de Caché:** Redis se utiliza para aliviar la carga de bases de datos relacionales almacenando resultados de consultas frecuentes o datos de acceso repetitivo, reduciendo drásticamente el tiempo de respuesta de la aplicación.
- **Carritos de Compra:** En aplicaciones de e-commerce, permite mantener carritos de compra vinculados al ID de usuario, asegurando que estén disponibles de forma instantánea a través de diferentes sesiones o dispositivos.
- **Analíticas y Tablas de Posiciones (Leaderboards) en tiempo real:** Gracias a estructuras como los **Conjuntos Ordenados (Sorted Sets)**, Redis es ideal para gestionar contadores de eventos, rankings de juegos o estadísticas que requieren actualizaciones masivas y constantes en tiempo real.
- **Broker de Mensajes y Streaming:** Funciona como un intermediario para la comunicación entre aplicaciones mediante el modelo **Pub/Sub** (Publicación/Suscripción) y la estructura **Streams**, facilitando el procesamiento de flujos de datos y eventos en tiempo real.
- **Perfiles de Usuario y Preferencias:** Permite almacenar objetos que contienen configuraciones personalizadas (idioma, zona horaria, colores) para que obtenerlas sea una operación de un solo paso.
- **Datos con Expiración Temporal:** Se utiliza para información volátil que debe desaparecer tras un tiempo determinado, como códigos de verificación, tokens de seguridad o avisos temporales, utilizando la propiedad `expiry_secs` (TTL).

- **Búsqueda Vectorial e IA:** Recientemente, Redis se ha posicionado como una **base de datos vectorial**, fundamental para almacenar contextos en tiempo real para aplicaciones de inteligencia artificial y modelos de lenguaje (LLMs).
 - **Registro de Eventos (Logging):** Debido a su alta capacidad de escritura, es apto para capturar logs de eventos de forma rápida antes de ser procesados o archivados.
-

Sección 2 - Manejo de Strings

2.1 Valores de texto

9. Agregar una clave package con el valor "Bariloche 3 days".

```
SET package "Bariloche 3 days"  
OK
```

10. Agregar una clave user con el valor "Turismo BD2". Obtener el valor de la clave user.

```
SET user "Turismo BD2"  
OK  
GET user  
"Turismo BD2"
```

11. Obtener todas las claves almacenadas actualmente.

```
KEYS *  
1) "package"  
2) "user"
```

12. Agregar una clave user con el valor "Cronos Turismo". ¿Cuál es el valor actual de la clave user?

```
SET user "Cronos Turismo"  
OK  
GET user  
"Cronos Turismo"
```

13. Concatenar " S.A." a la clave user. ¿Cuál es el valor actual de la clave user?


```
APPEND user " S.A."
(integer) 19
GET user
"Cronos Turismo S.A."
```

14. Eliminar la clave user. ¿Qué valor retorna si se intenta obtener la clave user luego de eliminarla?

```
DEL user
(integer) 1
GET user
(nil)
```

2.2 Valores numéricos

15. Verificar si existe la clave visits.

```
EXISTS visits
(integer) 0
```

16. Agregar una clave visits con el valor 0.

```
SET visits 0
OK
```

17. Incrementar en 1 la clave visits. ¿Cuál es el valor actual?

```
INCR visits
(integer) 1
GET visits
"1"
```

18. Incrementar en 5 la clave visits. ¿Cuál es el valor actual?

```
INCRBY visits 5
(integer) 6
GET visits
"6"
```

19. Decrementar en 1 la clave visits. ¿Cuál es el valor actual?

```
DECR visits
(integer) 5
GET visits
"5"
```

20. Incrementar en 2 la clave visits. ¿Cuál es el valor actual?

```
INCRBY visits 2
(integer) 7
GET visits
"7"
```

21. Agregar una clave "value package" con el valor 539789.32.

```
SET "value package" 539789.32
OK
```

22. Incrementar en 20000 la clave "value package". ¿Cuál es el valor actual?

```
INCRBYFLOAT "value package" 20000
"559789.3199999999999999318"
GET "value package"
"559789.3199999999999999318"
```

23. ¿Cual es el tipo de datos de "value package", visits y user?

```
TYPE "value package"
string
TYPE visits
string
TYPE user
none
```

Sección 3 - Manejo de Claves

24. Obtener todas las claves que empiecen con "v".

```
KEYS v*
1) "value package"
```

```
2) "visits"
```

25. Obtener todas las claves que contengan la letra "t".

```
KEYS *t*  
1) "visits"
```

26. Obtener todas las claves que terminan con "age".

```
KEYS *age  
1) "value package"  
2) "package"
```

27. Renombrar la clave "package" por "bariloche package".

```
RENAME package "bariloche package"  
OK
```

28. ¿Qué comando se utiliza para renombrar una clave solo si el nombre destino no existe aún?

```
RENAMENX package "bariloche package"  
→ (integer) 1 si renombró  
→ (integer) 0 si el destino ya existe
```

29. Eliminar todas las claves.

```
FLUSHDB  
OK
```

Sección 4 - Expiración de Claves

30. Agregar una clave agency con el valor "Cronos Tours".

```
SET agency "Cronos Tours"  
OK
```

31. ¿Cuál es el tiempo de vida (TTL) de la clave agency?

```
TTL agency
(integer) -1
```

-1 significa que la clave existe pero no tiene expiración configurada.

31. Agregar una expiración de 30 segundos a la clave agency.

```
EXPIRE agency 30
(integer)
```

32. ¿Cuál es el tiempo de vida de la clave agency luego de agregar la expiración?

```
TTL agency
(integer) 19
```

33. Pasados los 30 segundos: ¿cuál es el TTL de agency? ¿Que retorna si se solicita el valor de agency?

```
TTL agency
(integer) -2
GET agency
(nil)
```

-2 significa que la clave ya no existe (expiró). GET retorna nil.

34. Agregar una clave agency con el valor "Cronos Tours" que expire en 20 segundos desde su creación.

```
SET agency "Cronos Tours" EX 20
OK
```

Sección 5 - Listas

36. Insertar una lista llamada pets con el valor "dog".

```
RPUSH pets "dog"
(integer) 1
```

37. ¿Qué sucede si se ejecuta el comando GET sobre pets? ¿Cómo se obtienen los valores de una lista?

```
GET pets
(error) WRONGTYPE Operation against a key holding the wrong kind of value
LRANGE pets 0 -1
1) "dog"

GET solo funciona con strings. Para listas se usa LRANGE. 0 -1 = todos los elementos.
```

38. Agregar a la lista pets el valor "cat" por la izquierda.

```
LPUSH pets "cat"
(integer) 2
LRANGE pets 0 -1
1) "cat"
2) "dog"
```

39. Agregar a la lista pets el valor "fish" por la derecha.

```
RPUSH pets "fish"
(integer) 3
LRANGE pets 0 -1
1) "cat"
2) "dog"
3) "fish"
```

40. ¿Qué tipo de dato es el valor de pets?

```
TYPE pets
list
```

41. Eliminar el valor del extremo izquierdo de la lista.

```
LPOP pets
"cat"

Remueve y retorna el elemento del extremo izquierdo.
```

42. Eliminar el valor del extremo derecho de la lista.

```
RPOP pets
"fish"
```

43. Agregar a una clave "vuelo:ar389" los valores: aep, mdz, brc, nqn y mdq.

```
RPUSH vuelo:ar389 aep mdz brc nqn mdq
(integer) 5
```

44. Ordenar los valores de la lista "vuelo:ar389". ¿Qué sucede si se solicitan todos los valores de la lista luego de ordenarla?

```
SORT vuelo:ar389 ALPHA
1) "aep"
2) "brc"
3) "mdq"
4) "mdz"
5) "nqn"
LRANGE vuelo:ar389 0 -1
6) "aep"
7) "mdz"
8) "brc"
9) "nqn"
10) "mdq"
```

SORT no modifica la lista original; solo devuelve el resultado ordenado. El orden interno se mantiene igual.

45. Insertar el valor "fte" inmediatamente después de "brc".

```
LINSERT vuelo:ar389 AFTER brc fte
(integer) 6
LRANGE vuelo:ar389 0 -1
1) "aep"
2) "mdz"
3) "brc"
4) "fte"
5) "nqn"
6) "mdq"
```

46. Insertar el valor "ush" inmediatamente antes de "fte".

```
LINSERT vuelo:ar389 BEFORE fte ush
(integer) 7
LRANGE vuelo:ar389 0 -1
1) "aep"
2) "mdz"
3) "brc"
4) "ush"
5) "fte"
6) "nqn"
7) "mdq"
```

47. Modificar el último elemento de la lista por "sla".

```
LSET vuelo:ar389 -1 sla
OK
LRANGE vuelo:ar389 0 -1
1) "aep"
2) "mdz"
3) "brc"
4) "ush"
5) "fte"
6) "nqn"
7) "sla"
```

Índice `-1` = último elemento. `LSET` reemplaza por posición.

48. Obtener la cantidad de elementos de "vuelo:ar389".

```
LLEN vuelo:ar389
(integer) 7
```

49. Obtener el tercer valor de "vuelo:ar389".

```
LINDEX vuelo:ar389 2
"brc"
```

Los índices en Redis empiezan en `0`. Posición `3` = índice `2`.

50. Eliminar el valor "aep" de "vuelo:ar389".

```
LREM vuelo:ar389 1 aep
(integer) 1
```

```
LRANGE vuelo:ar389 0 -1
```

```
1) "mdz"  
2) "brc"  
3) "ush"  
4) "fte"  
5) "nqn"  
6) "sla"
```

LREM count valor: count=1 elimina la primera ocurrencia. count=0 elimina todas.

51. Quedarse únicamente con los valores de las posiciones 3 a 5 de "vuelo:ar389".

```
LTRIM vuelo:ar389 2 4
```

```
OK
```

```
LRANGE vuelo:ar389 0 -1
```

```
1) "ush"  
2) "fte"  
3) "nqn"
```

LTRIM índice_inicio índice_fin (base 0). Posiciones 3-5 = índices 2-4. Descarta el resto.

52. Agregar en "vuelo:ar389" el valor "fte". ¿Cuántas veces aparece ahora en la lista?

```
RPUSH vuelo:ar389 fte
```

```
(integer) 4
```

```
LRANGE vuelo:ar389 0 -1
```

```
1) "ush"  
2) "fte"  
3) "nqn"  
4) "fte"
```

```
LPOS vuelo:ar389 fte RANK 1 COUNT 0
```

```
5) (integer) 1  
6) (integer) 3
```

Las listas en Redis admiten duplicados. "fte" aparece 2 veces: en el índice 2 y en el 3. LPOS con COUNT 0 devuelve todas las posiciones.

Sección 6 - Conjuntos (Sets)

53. Agregar un conjunto llamado airports con los siguientes valores: eze aep nqn mdz mdq ush fte sla aep nqn brc cpc juju aep tuc eqs


```
SADD airports eze aep nqn mdz mdq ush fte sla aep nqn brc cpc juj aep tuc eqs
(integer) 13
```

Redis acepta todos los valores pero solo almacena únicos. Los duplicados (aep x3, nqn x2) son ignorados.

54. ¿Cuántos valores tiene el conjunto? ¿Por qué puede diferir de la cantidad de valores ingresados?

```
SCARD airports
(integer) 13
```

Se ingresaron 16 valores pero aep apareció 3 veces y nqn 2 veces. Los Sets no admiten duplicados, por eso quedan 13 únicos.

55. Listar los valores del conjunto airports.

```
SMEMBERS airports
```

```
1) "eze"
2) "aep"
3) "nqn"
4) "mdz"
5) "mdq"
6) "ush"
7) "fte"
8) "sla"
9) "brc"
10) "cpc"
11) "juj"
12) "tuc"
13) "eqs"
```

56. Quitar el valor "cpc" del conjunto airports.

```
SREM airports cpc
(integer) 1
```

Retorna 1 si existía y fue removido, 0 si no existía.

56. Quitar un valor aleatorio del conjunto airports.

```
SPOP airports
```

```
"brc"
```

SPOP elimina y devuelve un elemento al azar.

56. ¿Qué cantidad de valores tiene airports ahora?

```
SCARD airports
```

```
(integer) 11
```

56. Comprobar si "cpc" es miembro del conjunto airports.

```
SISMEMBER airports cpc
```

```
(integer) 0
```

0 = no es miembro, 1 = sí es miembro.

56. Mover los valores "sla" y "juj" a un nuevo conjunto denominado noa_airports.

```
SMOVE airports noa_airports sla
```

```
(integer) 1
```

```
SMOVE airports noa_airports juj
```

```
(integer) 1
```

SMOVE origen destino valor. Atómicamente quita del origen y agrega al destino.

57. Obtener la unión de los conjuntos airports y noa_airports. ¿Modifica los conjuntos originales?

```
SUNION airports noa_airports
```

```
1) "sla"
```

```
2) "mdz"
```

```
3) "aep"
```

```
4) "eze"
```

```
5) "ush"
```

```
6) "juj"
```

```
7) "mdq"
```

```
8) "fte"
```

```
9) "tuc"
```

```
10) "nqn"
```

```
11) "eqs"
```

SUNION solo devuelve el resultado; no modifica ningún conjunto original.

57. Realizar la unión de airports y noa_airports y almacenar el resultado en un nuevo conjunto llamado total_airports.

```
SUNIONSTORE total_airports airports noa_airports  
(integer) 11
```

SUNIONSTORE destino key1 key2. Persiste el resultado en un nuevo conjunto.

58. Realizar la intersección entre total_airports y noa_airports.

```
SINTER total_airports noa_airports  
1) "sla"  
2) "juj"
```

59. Realizar la diferencia entre total_airports y noa_airports.

```
SDIFF total_airports noa_airports  
1) "nqn"  
2) "ush"  
3) "eze"  
4) "mdz"  
5) "aep"  
6) "mdq"  
7) "fte"  
8) "tuc"  
9) "eqs"
```

Sección 7 - Conjuntos Ordenados (Sorted Sets)

65. Agregar a un conjunto ordenado llamado passengers los siguientes datos (score nombre):

```
2.5 federico 4 alejandra 3 julian 1 ivan 2 tomas 2 luciana 2.4 natalia
```

```
ZADD passengers 2.5 federico 4 alejandra 3 julian 1 ivan 2 tomas 2 luciana 2.4  
natalia  
(integer) 7
```

Formato: ZADD key score member. El Sorted Set se ordena automáticamente por score de menor a mayor.

66. Obtener los valores del conjunto passengers.

```
ZRANGE passengers 0 -1
```

```
1) "ivan"  
2) "luciana"  
3) "tomas"  
4) "natalia"  
5) "federico"  
6) "julian"  
7) "alejandra"
```

Ordenados por score ascendente. Sin WITH SCORES no muestra los scores.

67. Actualizar el score de luciana a 2.7.

```
ZADD passengers 2.7 luciana  
(integer) 0
```

Si el miembro ya existe, ZADD actualiza su score. Retorna 0 porque no agregó un elemento nuevo.

68. Agregar al conjunto passengers a silvia con score 5.1.

```
ZADD passengers 5.1 silvia  
(integer) 1
```

69. Incrementar en 2 el score de alejandra.

```
ZINCRBY passengers 2 alejandra  
"6"
```

70. Obtener los valores del conjunto passengers con sus scores.

```
ZRANGE passengers 0 -1 WITHSCORES
```

```
1) "ivan"  
2) "1"  
3) "tomas"  
4) "2"  
5) "natalia"
```

```
6) "2.4"
7) "federico"
8) "2.5"
9) "luciana"
10) "2.7"
11) "julian"
12) "3"
13) "silvia"
14) "5.1"
15) "alejandra"
16) "6"
```

71. Obtener los valores del conjunto passengers con sus scores en orden inverso.

```
ZRANGE passengers 0 -1 WITHSCORES REV
```

```
1) "alejandra"
2) "6"
3) "silvia"
4) "5.1"
5) "julian"
6) "3"
7) "luciana"
8) "2.7"
9) "federico"
10) "2.5"
11) "natalia"
12) "2.4"
13) "tomas"
14) "2"
15) "ivan"
16) "1"
```

REV invierte el orden. También se puede usar ZREVRANGE (comando legacy).

72. Obtener la cantidad de elementos del conjunto passengers.

```
ZCARD passengers
(integer) 8
```

73. Obtener la cantidad de elementos que tienen scores entre 2 y 3.

```
ZCOUNT passengers 2 3
(integer) 5
```

Los extremos son inclusivos

74. Obtener el ranking de julian en el conjunto passengers.

```
ZRANK passengers julian  
(integer) 5
```

Base 0, orden ascendente. julian tiene el 6to menor score → índice 5.

75. Obtener el score de tomas en el conjunto passengers.

```
ZSCORE passengers tomas  
"2"
```

76. Extraer el elemento con menor score del conjunto passengers.

```
ZPOPMIN passengers  
1) "ivan"  
2) "1"
```

ZPOPMIN elimina y retorna el elemento con el score más bajo.

77. Extraer el elemento con mayor score del conjunto passengers.

```
ZPOPMAX passengers  
1) "alejandra"  
2) "6"
```

ZPOPMAX elimina y retorna el elemento con el score más alto.

78. Eliminar del conjunto passengers al valor silvia.

```
ZREM passengers silvia  
(integer) 1
```

Sección 8 - Hashes

79. Agregar a un hash llamado user:cronos los siguientes campos: "razon social" "cronos s.a" domicilio "47 236 La Plata" "telefono" 2215556677

```
HSET user:cronos "razon social" "cronos s.a" domicilio "47 236 La Plata" telefono 2215556677
(integer) 3
```

HSET acepta múltiples pares campo-valor. Los campos con espacios van entre comillas.

80. Agregar el campo mail con el valor info@cronos.com.ar al hash user:cronos.

```
HSET user:cronos mail info@cronos.com.ar
(integer) 1
```

81. Obtener todos los campos y valores de user:cronos.

```
HGETALL user:cronos
1) "razon social"
2) "cronos s.a"
3) "domicilio"
4) "47 236 La Plata"
5) "telefono"
6) "2215556677"
7) "mail"
8) "info@cronos.com.ar"
```

82. Obtener únicamente el valor del campo mail de user:cronos.

```
HGET user:cronos mail
"info@cronos.com.ar"
```

83. Eliminar el campo teléfono de user:cronos.

```
HDEL user:cronos telefono
(integer) 1
```

84. Obtener la cantidad de campos de user:cronos.

```
HLEN user:cronos
```

```
(integer) 3
```

85. Obtener las claves (nombres de campos) de user:cronos.

```
HKEYS user:cronos
1) "razon social"
2) "domicilio"
3) "mail"
```

86. Determinar si existe el campo cuil en user:cronos.

```
HEXISTS user:cronos cuil
(integer) 0

0 = no existe, 1 = existe.
```

87. Obtener todos los valores (sin los nombres de campos) de user:cronos.

```
HVALS user:cronos
1) "cronos s.a"
2) "47 236 La Plata"
3) "info@cronos.com.ar"
```

88. Obtener la longitud del valor del campo mail de user:cronos.

```
HSTRLEN user:cronos mail
(integer) 18
```

Sección 9 - Geospacial

89. Agregar en un conjunto denominado cities las siguientes localidades con sus coordenadas (longitud, latitud):

Ciudad	Latitud	Longitud
Buenos Aires	-34.61315	-58.37723
Cordoba	-31.41350	-64.18105
Rosario	-32.94682	-60.63932

Ciudad	Latitud	Longitud
Mendoza	-32.89084	-68.82717
San Miguel de Tucuman	-26.82414	-65.22260
La Plata	-34.92145	-57.95453
Mar del Plata	-38.00042	-57.55620
Salta	-24.78590	-65.41166
Santa Fe	-31.64881	-60.70868
San Juan	-31.53750	-68.53639
Resistencia	-27.46056	-58.98389
Santiago del Estero	-27.79511	-64.26149
Posadas	-27.36708	-55.89608
San Salvador de Jujuy	-24.19457	-65.29712
Bahia Blanca	-38.71959	-62.27243
Parana	-31.73271	-60.52897

```
GEOADD cities -58.37723 -34.61315 "Buenos Aires" -64.18105 -31.41350 "Cordoba"
-60.63932 -32.94682 "Rosario" -68.82717 -32.89084 "Mendoza" -65.22260 -26.82414
"San Miguel de Tucuman" -57.95453 -34.92145 "La Plata" -57.55620 -38.00042 "Mar del
Plata" -65.41166 -24.78590 "Salta" -60.70868 -31.64881 "Santa Fe" -68.53639
-31.53750 "San Juan" -58.98389 -27.46056 "Resistencia" -64.26149 -27.79511
"Santiago del Estero" -55.89608 -27.36708 "Posadas" -65.29712 -24.19457 "San
Salvador de Jujuy" -62.27243 -38.71959 "Bahia Blanca" -60.52897 -31.73271 "Parana"
(integer) 16
```

90. Obtener todos los miembros del conjunto cities.

```
ZRANGE cities 0 -1
1) "Mendoza"
2) "San Juan"
3) "Bahia Blanca"
4) "Mar del Plata"
5) "Buenos Aires"
6) "La Plata"
7) "Rosario"
8) "Cordoba"
9) "San Miguel de Tucuman"
10) "Santiago del Estero"
11) "Parana"
```

- 12) "Santa Fe"
- 13) "Resistencia"
- 14) "Salta"
- 15) "San Salvador de Jujuy"
- 16) "Posadas"

Los datos geo se almacenan como Sorted Set. ZRANGE lista los miembros por su score interno (que codifica las coordenadas).

91. Obtener las coordenadas almacenadas de Santa Fe.

```
GEOPOS cities "Santa Fe"  
1) 1) "-60.708681643009186"  
   2) "-31.64881101691541"
```

Redis devuelve longitud, latitud. Hay mínima imprecisión por la codificación interna.

92. Obtener la distancia en kilómetros entre Buenos Aires y Córdoba.

```
GEODIST cities "Buenos Aires" "Cordoba" km  
"647.6303"
```

Unidades disponibles: m, km, mi, ft.

93. Obtener las ciudades que se encuentran en un radio de 100 km de la coordenada (-27.37, -55.9), incluyendo la distancia de cada una.

```
GEOSEARCH cities FROMLONLAT -55.9 -27.37 BYRADIUS 100 km ASC WITHCOORD WITHDIST  
1) 1) "Posadas"  
   2) "0.5055"  
3) 1) "-55.89607983827591"  
   2) "-27.367078923482953"
```

FROMLONLAT recibe longitud primero. ASC ordena por distancia. Solo Posadas queda dentro del radio.

94. Obtener las ciudades que se encuentran a menos de 700 km de Córdoba.

```
GEOSEARCH cities FROMMEMBER "Cordoba" BYRADIUS 700 km ASC WITHDIST  
1) 1) "Cordoba"  
   2) "0.0000"
```

- 2) 1) "Santa Fe"
2) "330.2202"
- 3) 1) "Parana"
2) "347.8771"
- 4) 1) "Rosario"
2) "374.4698"
- 5) 1) "Santiago del Estero"
2) "402.5353"
- 6) 1) "San Juan"
2) "413.3545"
- 7) 1) "Mendoza"
2) "467.3004"
- 8) 1) "San Miguel de Tucuman"
2) "520.3835"
- 9) 1) "Buenos Aires"
2) "647.6303"
- 10) 1) "Resistencia"
2) "668.2173"
- 11) 1) "La Plata"
2) "698.5502"

FROMMEMBER usa una ciudad ya cargada como punto de origen. El resultado puede variar levemente por precisión de coordenadas.